

Artificial Intelligence

From RAG to fabric: Lessons learned from building real-world RAGs at GenAIIC – Part 2

by Aude Genevay | on 15 NOV 2024 | in [Advanced \(300\)](#), [Amazon Bedrock](#), [Artificial Intelligence](#), [Technical How-to](#), [Thought Leadership](#) | [Permalink](#) | [Comments](#) | [Share](#)

In [Part 1](#) of this series, we defined the Retrieval Augmented Generation (RAG) framework to augment large language models (LLMs) with a text-only knowledge base. We gave practical tips, based on hands-on experience with customer use cases, on how to improve text-only RAG solutions, from optimizing the retriever to mitigating and detecting hallucinations.

This post focuses on doing RAG on heterogeneous data formats. We first introduce routers, and how they can help managing diverse data sources. We then give tips on how to handle tabular data and will conclude with multimodal RAG, focusing specifically on solutions that handle both text and image data.

Overview of RAG use cases with heterogeneous data formats

After a first wave of text-only RAG, we saw an increase in customers wanting to use a variety of data for Q&A. The challenge here is to retrieve the relevant data source to answer the question and correctly extract information from that data source. Use cases we have worked on include:

- **Technical assistance for field engineers** – We built a system that aggregates information about a company's specific products and field expertise. This centralized system consolidates a wide range of data sources, including detailed reports, FAQs, and technical documents. The system integrates structured data, such as tables containing product properties and specifications, with unstructured text documents that provide in-depth product descriptions and usage guidelines. A chatbot enables field engineers to quickly access relevant information, troubleshoot issues more effectively, and share knowledge across the organization.
- **Oil and gas data analysis** – Before beginning operations at a well, an oil and gas company will collect and process a diverse range of data to identify potential reservoirs, assess risks, and optimize drilling strategies. The data sources may include seismic surveys, well logs, core samples, geochemical analyses, and production histories, with some of it in industry-specific formats. Each category necessitates specialized generative AI-powered tools to generate insights. We built a chatbot that can answer questions across this complex data landscape, so that oil and gas companies can make faster and more informed decisions, improve exploration success rates, and decrease time to first oil.
- **Financial data analysis** – The financial sector uses both unstructured and structured data for market analysis and decision-making. Unstructured data includes news articles, regulatory filings, and social media, providing qualitative insights. Structured data consists of stock prices, financial statements, and economic indicators. We built a RAG system that combines these diverse data types into a single knowledge base, allowing analysts to efficiently access and correlate information. This approach enables nuanced analysis by combining numerical trends with textual insights to identify opportunities, assess risks, and forecast market movements.
- **Industrial maintenance** – We built a solution that combines maintenance logs, equipment manuals, and visual inspection data to optimize maintenance schedules and troubleshooting. This multimodal approach integrates written reports and procedures with images and diagrams of machinery, allowing maintenance technicians to quickly access both descriptive information and visual representations of equipment. For example, a technician could query the system about a specific machine part, receiving both textual maintenance history and annotated images showing wear patterns or common failure points, enhancing their ability to diagnose and resolve issues efficiently.

- **Ecommerce product search** – We built several solutions to enhance the search capabilities on ecommerce websites to improve the shopping experience for customers. Traditional search engines rely mostly on text-based queries. By integrating multimodal (text and image) RAG, we aimed to create a more comprehensive search experience. The new system can handle both text and image inputs, allowing customers to upload photos of desired items and receive precise product matches.

Using a router to handle heterogeneous data sources

In RAG systems, a router is a component that directs incoming user queries to the appropriate processing pipeline based on the query's nature and the required data type. This routing capability is crucial when dealing with heterogeneous data sources, because different data types often require distinct retrieval and processing strategies.

Consider a financial data analysis system. For a qualitative question like “What caused inflation in 2023?”, the router would direct the query to a text-based RAG that retrieves relevant documents and uses an LLM to generate an answer based on textual information. However, for a quantitative question such as “What was the average inflation in 2023?”, the router would direct the query to a different pipeline that fetches and analyzes the relevant dataset.

The router accomplishes this through intent detection, analyzing the query to determine the type of data and analysis required to answer it. In systems with heterogeneous data, this process makes sure each data type is processed appropriately, whether it's unstructured text, structured tables, or multimodal content. For instance, analyzing large tables might require prompting the LLM to generate Python or SQL and running it, rather than passing the tabular data to the LLM. We give more details on that aspect later in this post.

In practice, the router module can be implemented with an initial LLM call. The following is an example prompt for a router, following the example of financial analysis with heterogeneous data. To avoid adding too much latency with the routing step, we recommend using a smaller model, such as Anthropic's Claude Haiku on [Amazon Bedrock](#).

```
<example>
<query>I need to see the latest 10-K filing for Amazon.</query>
<data_source>Regulatory Filings Database</data_source>
<reason>SEC 10K which are in Regulatory Filings database.</reason>
</example>

<example>
<query>What's the best restaurant in town?</query>
<data_source>None</data_source>
<reason>Restaurant recommendations are not related to any data source.</reason>
</example>

Here is the user query
<query>
{user_query}
</query>

Output the data source in <data_source> tags and the explanation in <reason> tags.
" " "
```

Prompting the LLM to explain the routing logic may help with accuracy, by forcing the LLM to “think” about its answer, and also for debugging purposes, to understand why a category might not be routed properly.

The prompt uses XML tags following Anthropic’s Claude best practices. Note that in this example prompt we used `<data_source>` tags but something similar such as `<category>` or `<label>` could also be used. Asking the LLM to also structure its response with XML tags allows us to parse out the category from the LLM answer, which can be done with the following code:

```
# Parse out the data source
pattern = r"<data_source>(.*?)</data_source>"
data_source = re.findall(
    pattern, llm_response, re.DOTALL
)[0]
```

From a user’s perspective, if the LLM fails to provide the right routing category, the user can explicitly ask for the data source they want to use in the query. For instance, instead of saying “What caused inflation in 2023?”, the user could disambiguate by asking “What caused inflation in 2023 according to analysts?”, and instead of “What was the average inflation in 2023?”, the user could ask “What was the average inflation in 2023? Look at the indicators.”

Another option for a better user experience is to add an option to ask for clarifications in the router, if the LLM finds that the query is too ambiguous. We can add this as an additional “data source” in the router using the following code:

```
<source>
<name>Clarifications</name>
<description>If the query is too ambiguous, use this to ask the user for more\
clarifications. Put your reply to the user in the reason tags</description>
</source>
```

We use an associated example:

```
<example>
<query>What's can you tell me about Amazon stock?</query>
<data_source>Clarifications</data_source>
<reason>I'm not sure how to best answer your question,\
do you want me to look into Stock Prices, Analyst Notes, Regulatory filings?</reason>
</example>
```

If in the LLM’s response, the data source is `Clarifications`, we can then directly return the content of the `<reason>` tags to the user for clarifications.

An alternative approach to routing is to use the native tool use capability (also known as function calling) available within the [Bedrock Converse API](#). In this scenario, each category or data source would be defined as a ‘tool’ within

the API, enabling the model to select and use these tools as needed. Refer to [this documentation](#) for a detailed example of tool use with the Bedrock Converse API.

Using LLM code generation abilities for RAG with structured data

Consider an oil and gas company analyzing a dataset of daily oil production. The analyst may ask questions such as “Show me all wells that produced oil on June 1st 2024,” “What well produced the most oil in June 2024?”, or “Plot the monthly oil production for well XYZ for 2024.” Each question requires different treatment, with varying complexity. The first one involves filtering the dataset to return all wells with production data for that specific date. The second one requires computing the monthly production values from the daily data, then finding the maximum and returning the well ID. The third one requires computing the monthly average for well XYZ and then generating a plot.

LLMs don’t perform well at analyzing tabular data when it’s added directly in the prompt as raw text. A simple way to improve the LLM’s handling of tables is to add it in the prompt in a more structured format, such as markdown or XML. However, this method will only work if the question doesn’t require complex quantitative reasoning and the table is small enough. In other cases, we can’t reliably use an LLM to analyze tabular data, even when provided as structured format in the prompt.

On the other hand, LLMs are notably good at code generation; for instance, [Anthropic’s Claude Sonnet 3.5 has 92% accuracy](#) on the [HumanEval](#) code benchmark. We can take advantage of that capability by asking the LLM to write Python (if the data is stored in a CSV, Excel, or Parquet file) or SQL (if the data is stored in a SQL database) code that performs the required analysis. Popular libraries Llama Index and LangChain both offer out-of-the-box solutions for text-to-SQL ([Llama Index](#), [LangChain](#)) and text-to-Pandas ([Llama Index](#), [LangChain](#)) pipelines for quick prototyping. However, for better control over prompts, code execution, and outputs, it might be worth writing your own pipeline. Out-of-the-box solutions will typically prompt the LLM to write Python or SQL code to answer the user’s question, then parse and run the code from the LLM’s response, and finally send the code output back to the LLM for a final answer.

Going back to the oil and gas data analysis use case, take the question “Show me all wells that produced oil on June 1st 2024.” There could be hundreds of entries in the dataframe. In that case, a custom pipeline that directly returns the code output to the UI (the filtered dataframe for the date of June 1st 2024, with oil production greater than 0) would be more efficient than sending it to the LLM for a final answer. If the filtered dataframe is large, the additional call might cause high latency and even risks causing hallucinations. Writing your custom pipelines also allows you to perform some sanity checks on the code, to verify, for instance, that the code generated by the LLM will not create issues (such as modify existing files or data bases).

The following is an example of a prompt that can be used to generate Pandas code for data analysis:

```
prompt_template = """
You are an AI assistant designed to answer questions from oil and gas analysts.
You have access to a Pandas dataframe df that contains daily production data for oil p

Here is a sample from df:
<df_sample>
{sample}
</df_sample>
```

Here is the analyst's question:

```
<question>
```

```
{question}
```

```
</question>
```

```
<instructions>
```

- Use <scratchpad> tags to think about what you are going to do.
 - Put your the code in <code> tags.
 - The dataframes may contain nans, so make sure you account for those in your code.
- To your code, the final variable should be named "result"

We can then parse the code out from the <code> tags in the LLM response and run it using exec in Python. The following code is a full example:

```
import boto3
import pandas as pd

# Import the csv into a DataFrame
df = pd.read_csv('stock_prices.csv')

# Create an Amazon Bedrock client
bedrock_client = boto3.client('bedrock')

# Define the prompt
user_query = "Show me all wells that produced oil on June 1st 2024"
prompt = prompt_template.format(sample = df.sample(5), question=user_query))

# Call Anthropic Claude Sonnet
request_body = json.dumps(
    {
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 1000,
        "messages": [
```

Because we explicitly prompt the LLM to store the final result in the result variable, we know it will be stored in the `local_vars` dictionary under that key, and we can retrieve it that way. We can then either directly return this result to the user, or send it back to the LLM to generate its final response. Sending the variable back to the user directly can be useful if the request requires filtering and returning a large dataframe, for instance. Directly returning the variable to the user removes the risk of hallucination that can occur with large inputs and outputs.

Multimodal RAG

An emerging trend in generative AI is multimodality, with models that can use text, images, audio, and video. In this post, we focus exclusively on mixing text and image data sources.

In an industrial maintenance use case, consider a technician facing an issue with a machine. To troubleshoot, they might need visual information about the machine, not just a textual guide.

In ecommerce, using multimodal RAG can enhance the shopping experience not only by allowing users to input images to find visually similar products, but also by providing more accurate and detailed product descriptions from visuals of the products.

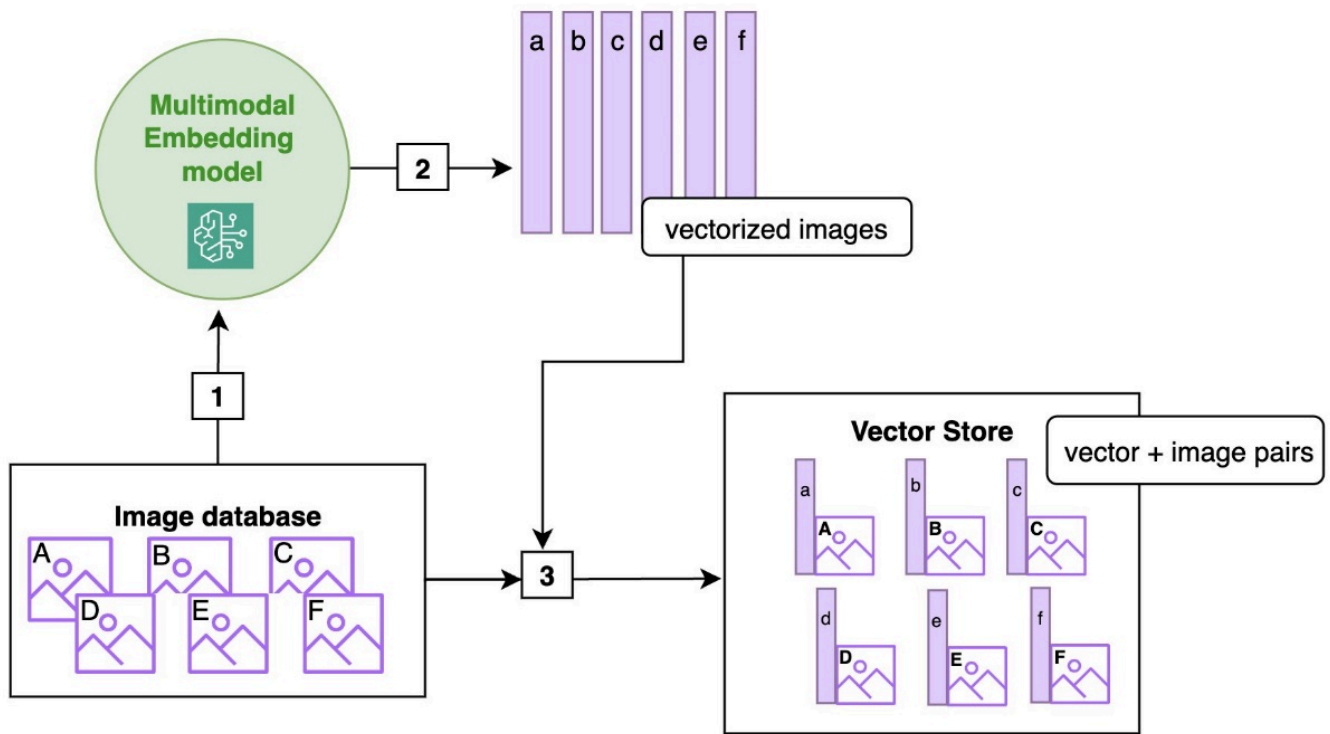
We can categorize multimodal text and image RAG questions in three categories:

- **Image retrieval based on text input** – For example:
 - “Show me a diagram to repair the compressor on the ice cream machine.”
 - “Show me red summer dresses with floral patterns.”
- **Text retrieval based on image input** – For example:
 - A technician might take a picture of a specific part of the machine and ask, “Show me the manual section for this part.”
- **Image retrieval based on text and image input** – For example:
 - A customer could upload an image of a dress and ask, “Show me similar dresses.” or “Show me items with a similar pattern.”

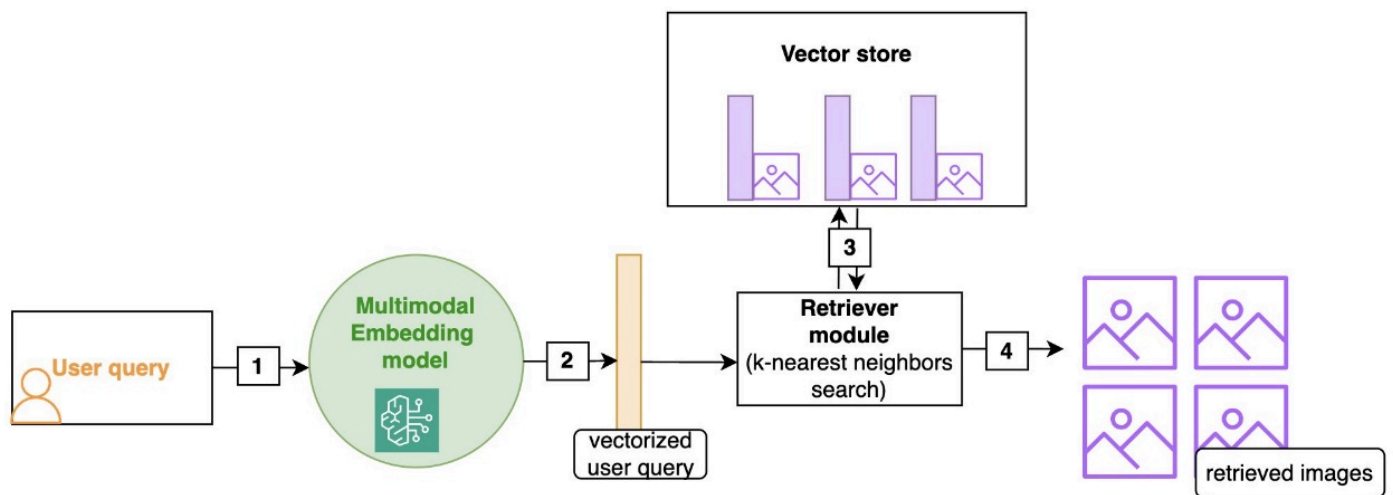
As with traditional RAG pipelines, the retrieval component is the basis of these solutions. Constructing a multimodal retriever requires having an embedding strategy that can handle this multimodality. There are two main options for this.

First, you could use a multimodal embedding model such as [Amazon Titan Multimodal Embeddings](#), which can embed both images and text into a shared vector space. This allows for direct comparison and retrieval of text and images based on semantic similarity. This simple approach is effective for finding images that match a high-level description or for matching images of similar items. For instance, a query like “Show me summer dresses” would return a variety of images that fit that description. It’s also suitable for queries where the user uploads a picture and asks, “Show me dresses similar to that one.”

The following diagram shows the ingestion logic with a multimodal embedding. The images in the database are sent to a multimodal embedding model that returns vector representations of the images. The images and the corresponding vectors are paired up and stored in the vector database.



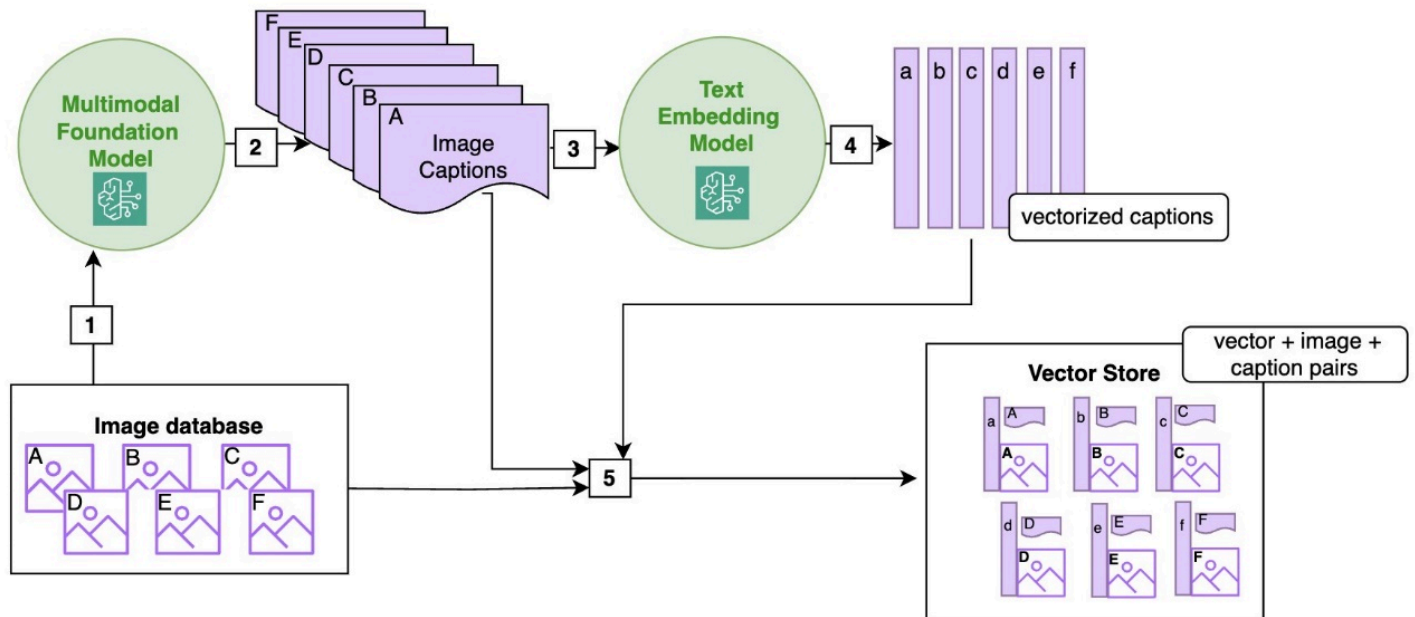
At retrieval time, the user query (which can be text or image) is passed to the multimodal embedding model, which returns a vectorized user query that is used by the retriever module to search for images that are close to the user query, in the embedding distance. The closest images are then returned.



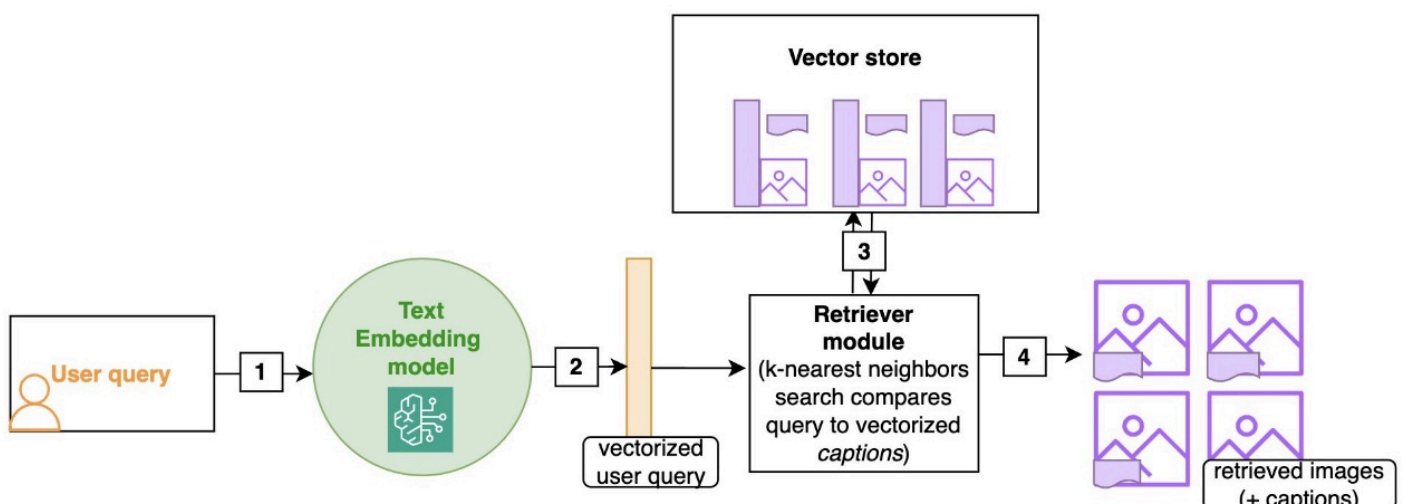
Alternatively, you could use a multimodal foundation model (FM) such as Anthropic's Claude v3 Haiku, Sonnet, or Opus, and Sonnet 3.5, all available on Amazon Bedrock, which can generate the caption of an image, which will then be used for retrieval. Specifically, the generated image description is embedded using a traditional text embedding (e.g. [Amazon Titan Embedding Text v2](#)) and stored in a vector store along with the image as metadata.

Captions can capture finer details in images, and can be guided to focus on specific aspects such as color, fabric, pattern, shape, and more. This would be better suited for queries where the user uploads an image and looks for similar items but only in some aspects (such as uploading a picture of a dress, and asking for skirts in a similar style). This would also work better to capture the complexity of diagrams in industrial maintenance.

The following figure shows the ingestion logic with a multimodal FM and text embedding. The images in the database are sent to a multimodal FM that returns image captions. The image captions are then sent to a text embedding model and converted to vectors. The images are paired up with the corresponding vectors and captions and stored in the vector database.



At retrieval time, the user query (text) is passed to the text embedding model, which returns a vectorized user query that is used by the retriever module to search for captions that are close to the user query, in the embedding distance. The images corresponding to the closest captions are then returned, optionally with the caption as well. If the user query contains an image, we need to use a multimodal LLM to describe that image similarly to the previous ingestion steps.



Example with a multimodal embedding model

The following is a code sample performing ingestion with Amazon Titan Multimodal Embeddings as described earlier. The embedded image is stored in an OpenSearch index with a k-nearest neighbors (k-NN) vector field.

```

from utils import *

# Read and encode the image
file_name = 'image.png'
image_base64 = read_and_encode_image(file_name)

# Embed the image using Amazon Titan Multimodal Embeddings
multi_embedding_model = "amazon.titan-embed-image-v1"
image_embedding = get_embedding(input = image_base64, model = multi_embedding_model)

# Get OpenSearch client (assume this function is available)
open_search = get_open_search_client()

# Create index in OpenSearch for storing embeddings
create_opensearch_index(name = 'multimodal-image-index', client = open_search)

# Index the image and its embedding in OpenSearch
request = {
    'image': image_base64,

```

The following is the code sample performing the retrieval with Amazon Titan Multimodal Embeddings:

```

# Use Amazon Titan Multimodal Embeddings to embed the user query
query_text = "Show me a diagram to repair the compressor on the ice cream machine."

query_embedding = get_embedding(input = image_base64, model = multi_embedding_model)

# Search for images that are close to that description in OpenSearch
search_query = {
    'query': {
        'bool': {
            'should': [
                {
                    'knn': {
                        'vector_field': {
                            'vector': text_embedding,
                            'k': 5
                        }
                    }
                }
            ]
        }
    }
}

```

In the response, we have the images that are closest to the user query in embedding space, thanks to the multimodal embedding.

Example with a multimodal FM

The following is a code sample performing the retrieval and ingestion described earlier. It uses Anthropic's Claude Sonnet 3 to caption the image first, and then Amazon Titan Text Embeddings to embed the caption. You could also use another multimodal FM such as Anthropic's Claude Sonnet 3.5, Haiku 3, or Opus 3 on Amazon Bedrock. The image, caption embedding, and caption are stored in an OpenSearch index. At retrieval time, we embed the user query using the same Amazon Titan Text Embeddings model and perform a k-NN search on the OpenSearch index to retrieve the relevant image.

```
# Read and encode the image
file_name = 'image.png'
image_base64 = read_and_encode_image(file_name)

# Use Anthropic Claude Sonnet to caption the image
caption = call_multimodal_llm(
    modelId="anthropic.claude-3-sonnet-20240229-v1:0",
    text="Describe this image in detail. Only output the description, nothing else"
    image = image_base64
)

# Compute text embedding for the caption
text_embedding_model = "amazon.titan-embed-text-v2:0"
caption_embedding = get_embedding(input = caption, model = text_embedding_model)

# Create the index with a mapping that has a knn vector field
open_search.indices.create(index='image-caption-index', body=mapping)
```

The following is code to perform the retrieval step using text embeddings:

```
# Compute embedding for a natural language query with text embedding
user_query= "Show me a diagram to repair the compressor on the ice cream machine."
query_embedding = get_embedding(input = caption, model = text_embedding_model)

# Search for images that match that query in OpenSearch
search_query = {
    'query': {
        'bool': {
            'should': [
                {
                    'knn': {
                        'vector_field': {
                            'vector': query_embedding,
                            'k': 5
                        }
                    }
                }
            ]
        }
    }
}
```

This returns the images whose captions are closest to the user query in the embedding space, thanks to the text embeddings. In the response, we get both the images and the corresponding captions for downstream use.

Comparative table of multimodal approaches

The following table provides a comparison between using multimodal embeddings and using a multimodal LLM for image captioning, across several key factors. Multimodal embeddings offer faster ingestion and are generally more cost-effective, making them suitable for large-scale applications where speed and efficiency are crucial. On the other hand, using a multimodal LLM for captions, though slower and less cost-effective, provides more detailed and customizable results, which is particularly useful for scenarios requiring precise image descriptions. Considerations such as latency for different input types, customization needs, and the level of detail required in the output should guide the decision-making process when selecting your approach.

.	Multimodal Embeddings	Multimodal LLM for Captions
Speed	Faster ingestion	Slower ingestion due to additional LLM call
Cost	More cost-effective	Less cost-effective
Detail	Basic comparison based on embeddings	Detailed captions highlighting specific features
Customization	Less customizable	Highly customizable with prompts
Text Input Latency	Same as multimodal LLM	Same as multimodal embeddings
Image Input Latency	Faster, no extra processing required	Slower, requires extra LLM call to generate image caption
Best Use Case	General use, quick and efficient data handling	Precise searches needing detailed image descriptions

Conclusion

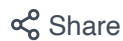
Building real-world RAG systems with heterogeneous data formats presents unique challenges, but also unlocks powerful capabilities for enabling natural language interactions with complex data sources. By employing techniques like intent detection, code generation, and multimodal embeddings, you can create intelligent systems that can understand queries, retrieve relevant information from structured and unstructured data sources, and provide coherent responses. The key to success lies in breaking down the problem into modular components and using the strengths of FMs for each component. Intent detection helps route queries to the appropriate processing logic, and code generation enables quantitative reasoning and analysis on structured data sources. Multimodal embeddings and multimodal FMs enable you to bridge the gap between text and visual data, enabling seamless integration of images and other media into your knowledge bases.

Get started with FMs and embedding models in [Amazon Bedrock](#) to build RAG solutions that seamlessly integrate tabular, image, and text data for your organization's unique needs.

About the Author



Aude Genevay is a Senior Applied Scientist at the Generative AI Innovation Center, where she helps customers tackle critical business challenges and create value using generative AI. She holds a PhD in theoretical machine learning and enjoys turning cutting-edge research into real-world solutions.



Comments

Log in to comment
